
HOSPITAL MANAGEMENT SYSTEM

TECHNICAL & FUNCTIONAL DOCUMENTATION

A Role-Based Web Platform for Hospital Operations Management

Prepared By	Arqam Hussain
Platform	Python 3.12 · Django 6.0.6 · SQLite
Version	1.0
Classification	Internal / Academic Use

Table of Contents

1. Executive Summary	3
2. Introduction.....	3
3. System Architecture	4
4. Technology Stack	6
5. Functional Overview	7
6. Data Model.....	8
7. Roles and Access Control	10
8. Core Workflows.....	11
9. Installation and Deployment Guide	11
10. Project Structure.....	13
11. Management Commands	13
12. Testing Strategy.....	14
13. Security Considerations.....	14
14. Limitations and Future Enhancements	15
15. Conclusion	15
16. Appendix — Glossary of Terms	16

1. Executive Summary

The Hospital Management System (HMS) is a full-featured, role-based web application built on the Django framework to digitize the day-to-day administrative and clinical operations of a small-to-mid-size hospital or clinic. It was conceived to address a very common and persistent operational problem: healthcare facilities of this size frequently continue to rely on paper registers, loose appointment slips, and disconnected spreadsheets to manage patients, doctors, and scheduling. This approach is slow, error-prone, difficult to audit, and offers no meaningful way to prevent scheduling conflicts or to retrieve a patient's history at the point of care.

HMS replaces that fragmented approach with a single, centralized, access-controlled platform. Every patient, doctor, department, appointment, prescription, and medical record is stored in one relational database and surfaced through role-appropriate dashboards, so that Administrators, Doctors, and Receptionists each see exactly the information and controls relevant to their responsibilities and nothing more. The system was designed with three guiding priorities in mind: data integrity (ensuring that relationships between patients, doctors, and appointments always remain consistent), operational safety (in particular, preventing a doctor from being double-booked for overlapping appointments), and usability (a clean, responsive Bootstrap-based interface that requires no specialized training to operate).

This document is a comprehensive technical and functional reference for the system. It walks through the problem the system solves, the architecture and technology choices behind it, the complete data model and the reasoning behind each design decision, the functional capabilities available to each user role, the installation and deployment procedure, and a candid discussion of the system's current limitations and the roadmap of enhancements that would be required to take it from an academic or demonstration deployment to a production-grade hospital information system.

2. Introduction

2.1 Background and Motivation

Small clinics and mid-size hospitals often cannot justify the cost or operational overhead of large, enterprise hospital information systems, yet they still face the same fundamental coordination problems as larger institutions: patients need to be registered once and recognized consistently thereafter; doctors need a schedule that reflects their true availability; appointments need to be booked without double-booking a doctor into two places at once; and every visit needs to produce a durable record — a prescription, a diagnosis, a set of test results — that can be retrieved later without having to ask the patient to repeat their history from memory. HMS was built specifically to close this gap: it is intentionally lightweight in its infrastructure requirements (it runs comfortably on a single SQLite database file with no external services required) while still implementing the core discipline of a proper hospital information system, namely structured data, enforced relationships, and role-based access control.

2.2 Purpose of This Document

The purpose of this document is to describe, in sufficient technical and functional depth, how the Hospital Management System is designed, how its internal components interact with one another, how it should be installed and configured, and how each of its features is intended to be used in practice. Where a particular design decision was made — for example, the choice of on-delete behavior for a given foreign key, or the decision to introduce a

custom user model before the first migration — this document explains not only what was done but why, so that a future maintainer can extend the system without inadvertently violating the assumptions it was built on.

2.3 Scope

The scope of this documentation extends across the entire application: the six cooperating Django applications that make up the system (accounts, departments, doctors, patients, appointments, and records), the relational schema that underlies them, the authentication and role-based access control layer, the appointment scheduling engine and its conflict-prevention logic, the prescription and medical-record subsystem, and the supporting operational tooling such as management commands for data seeding and automated e-mail reminders. It also covers deployment considerations, testing strategy, security posture, and a forward-looking set of recommendations for hardening the system for production use.

2.4 Objectives

The system was designed around five explicit objectives. First, to eliminate manual, paper-based hospital record-keeping in favor of a centralized digital system of record that every authorized staff member can query in real time. Second, to enforce secure, role-based access so that each class of user — administrative staff, clinical staff, and front-desk staff — interacts only with the functionality that is relevant and appropriate to their job function, reducing both accidental misuse and the surface area for malicious action. Third, to prevent scheduling conflicts by validating, at the moment an appointment is created, that the requested doctor is not already committed to another patient at an overlapping date and time. Fourth, to give doctors and administrative staff rapid, structured access to a patient's appointment history, prescriptions, and medical records, rather than requiring them to reconstruct that history from memory or from disconnected paper files. Fifth, to support basic operational reporting — including downloadable PDF prescriptions and automated e-mail reminders for upcoming appointments — so that the system provides value beyond simple record storage.

2.5 Intended Audience

This document is written for several overlapping audiences. Software engineers who need to extend or maintain the system will find the architecture, data model, and design-rationale sections most relevant. Quality-assurance reviewers validating the system's behavior will find the functional overview, core workflow descriptions, and testing strategy sections directly actionable. Technical evaluators and academic reviewers assessing the project as a piece of engineering work will find the executive summary, architecture, and limitations sections useful for forming a holistic judgment of the system's maturity. Finally, system administrators responsible for installing and operating the application will find the installation, deployment, and security sections written specifically to support that task.

3. System Architecture

HMS follows Django's Model-View-Template (MVT) architectural pattern, which is Django's variant of the more widely known Model-View-Controller pattern. In this pattern, the Model layer is responsible for defining the shape of the data and the rules that govern it; the View layer is responsible for processing requests, applying business logic and permission checks, and deciding what data to send back to the user; and the Template layer is responsible purely for presentation — rendering that data into HTML without embedding business logic. This separation of concerns is not merely academic: it means that a change to how an appointment conflict is detected

lives entirely inside the model or view layer and never requires touching a template, and conversely that a change to the visual design of the appointment list never risks breaking the underlying scheduling logic.

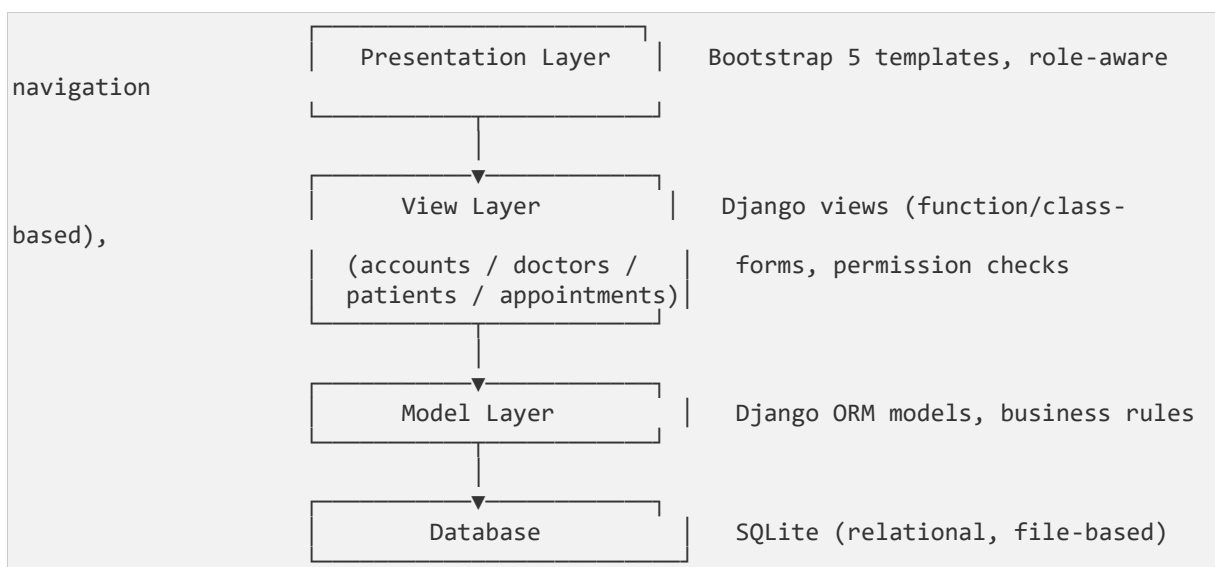
Beyond the MVT pattern itself, the application is further decomposed into six independent Django "apps," each of which owns exactly one functional domain of the hospital's operations. This modular decomposition was a deliberate architectural choice rather than an accident of how the code happened to grow. Keeping departments, doctors, patients, appointments, and records as separate apps means that each one can be reasoned about, tested, and modified largely in isolation, and it also mirrors the real-world organizational boundaries of a hospital, which makes the codebase easier for a new developer or reviewer to navigate intuitively.

3.1 Application Modules

The accounts application is the foundation of the system: it defines a custom User model that carries a role attribute in addition to the standard authentication fields, and it is responsible for login, logout, session handling, and for routing an authenticated user to the dashboard appropriate to their role. The departments application manages the directory of hospital departments — such as Cardiology, Pediatrics, or General Medicine — and provides full create, read, update, and delete (CRUD) functionality for administrators. The doctors application manages doctor profiles, including their department assignment, specialization, availability windows, and consultation fees. The patients application manages patient registration and profile data, along with search functionality by name, identifier, or phone number. The appointments application implements the scheduling engine itself, including the logic that prevents a doctor from being double-booked, and it manages the appointment status lifecycle. The records application manages the two clinical output types of a completed visit — prescriptions, which may contain multiple itemized medicine entries, and medical records, which capture diagnosis, treatment, and test results. Finally, the hospital application is the root Django project configuration, holding the global settings module, the top-level URL routing table, and the WSGI and ASGI entry points used to serve the application.

3.2 Architectural Diagram

The diagram below illustrates the layered request flow through the system, from the browser-rendered presentation layer down to the persistent data store, and identifies which architectural layer is responsible for which category of concern.



3.3 Design Principles

Four principles were applied consistently across the codebase. The first is single responsibility at the application level: each Django app owns exactly one functional domain, so that a change confined to, for instance, how prescriptions are itemized never needs to touch the appointment-scheduling logic. The second is fail-safe default access: every view that displays or modifies sensitive data is guarded by a role-checking decorator that denies access unless the authenticated user's role explicitly permits the action, meaning that a bug in a decorator would cause a false rejection rather than an unintended information leak. The third principle is data integrity first: rather than accepting Django's default cascading-delete behavior everywhere, each foreign-key relationship was reviewed individually and given a deletion policy — cascade or nullify — appropriate to whether the child record's meaning is entirely dependent on its parent. The fourth is progressive enhancement in the interface: every page functions correctly using plain server-rendered HTML and standard form submissions, with JavaScript layered on top purely to improve the experience, rather than being a prerequisite for core functionality.

4. Technology Stack

The technology choices behind HMS were made to keep the system easy to install, easy to reason about, and inexpensive to operate, while still being built on a framework — Django — that is mature, well-documented, and capable of scaling to a production-grade deployment if the underlying database and hosting configuration are upgraded accordingly.

Layer	Technology	Purpose
Programming Language	Python 3.12	Core application logic
Web Framework	Django 6.0.6	MVT framework, ORM, routing, forms, admin
Database	SQLite	Relational data store (file-based, zero-configuration)
Frontend	Bootstrap 5, HTML5, CSS3, vanilla JavaScript	Responsive, mobile-friendly interface
Document Generation	xhtml2pdf	Server-side rendering of prescriptions to PDF
Test Automation	Playwright	End-to-end browser testing
Process Definition	Procfile	Declares the web process for platform-as-a-service deployment

Python and Django were chosen as the core of the stack because Django's "batteries-included" philosophy provides authentication, an object-relational mapper, a form-validation system, and an administrative interface out of the box, which allowed development effort to be concentrated on hospital-specific business logic rather than on re-implementing infrastructure that a mature framework already solves well. SQLite was chosen for the data layer at this stage of the project specifically because it requires no separate database server process, which keeps the installation procedure to a handful of commands and makes the system trivially portable between machines; the trade-off, discussed later in the limitations section, is that SQLite is not intended for high-concurrency production workloads. Bootstrap 5 was selected for the frontend because it provides a consistent, responsive, accessible component library without requiring a separate JavaScript build pipeline, which keeps the frontend approachable for a Django-rendered, server-first application.

5. Functional Overview

5.1 Core Modules

The role-based access control module is the gatekeeper for the rest of the system: at login, each user is authenticated against the custom User model and routed to one of three dashboards — Administrator, Doctor, or Receptionist — each of which exposes a distinct set of navigation options and permitted actions. This is not simply a matter of hiding menu items in the template; every underlying view independently re-validates the user's role before performing any read or write operation, so that access control cannot be bypassed by directly requesting a URL that the interface does not display.

Patient management allows front-desk and administrative staff to register new patients, view and edit existing patient profiles, and search the patient directory. Each patient record aggregates, in one place, that patient's complete appointment history, all prescriptions issued to them, and all medical records associated with their visits, so that a doctor seeing a returning patient can review their history without needing to consult a separate system or a paper file.

Doctor management allows an administrator to add and edit doctor profiles, assign each doctor to a department, and configure their availability windows and consultation fee. This information feeds directly into the appointment-scheduling engine, which uses a doctor's declared availability and existing bookings to determine whether a new appointment request can be accepted.

Department management provides full CRUD functionality over the hospital's department directory, including the ability to designate a head of department from among the doctors assigned to it. Departments serve as the organizing structure that both doctors and, indirectly, appointments are grouped under, which supports department-level filtering and reporting elsewhere in the system.

Appointment scheduling is the operational heart of the system. When a receptionist or administrator books an appointment, the scheduling engine checks the selected doctor's existing appointments for the requested date and time slot before allowing the booking to proceed, which prevents the doctor from being committed to two patients at once. Once created, an appointment moves through a controlled status lifecycle — Pending, Confirmed, Completed, or Cancelled — that reflects its real-world progress and gates which further actions, such as prescription creation, are permitted.

Prescriptions are created by a doctor once an appointment has reached Completed status, ensuring that a prescription can never exist for a visit that did not actually occur. Each prescription can contain multiple itemized medicine entries, with each item specifying the medicine name, dosage, frequency, duration, and any special administration instructions, which allows a single prescription to represent a complete, multi-drug treatment plan rather than a single line of text.

Medical records complement prescriptions by capturing the clinical narrative of a visit — diagnosis, treatment administered, and any test results — and are likewise tied to a specific completed appointment, keeping the clinical record anchored to a verifiable event rather than existing as a free-floating note.

Search and filtering functionality is available throughout the system: patients can be located by name, identifier, or phone number; appointments can be filtered by date, doctor, department, or status; and doctors can be filtered by department or specialization. Every list view in the system is paginated at twenty records per page, which keeps page-load times predictable and consistent as the underlying dataset grows, and the entire interface is built on Bootstrap 5 so that it remains usable on desktop, tablet, and mobile viewports alike.

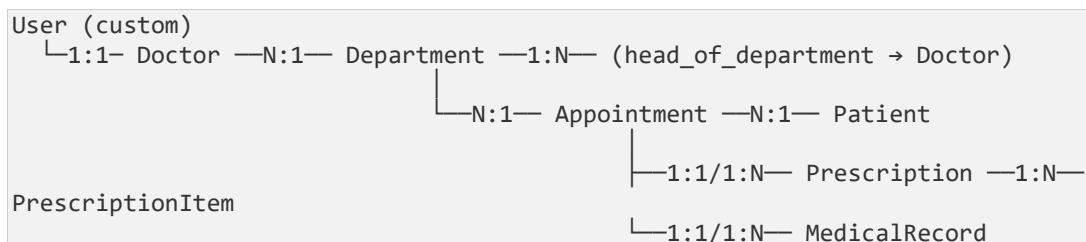
5.2 Extended Capabilities

Beyond the core record-keeping functionality, HMS includes several capabilities that extend it from a pure data-entry tool into a system with genuine day-to-day operational value. Prescriptions can be rendered and downloaded as a formatted PDF document via the `xhtml2pdf` library, giving patients a physical or digital document they can take to a pharmacy without requiring the pharmacy to have any access to the hospital's internal system. A scheduled management command dispatches automated e-mail reminders for appointments occurring the following day, reducing missed appointments without requiring staff to manually track and contact each patient. An aggregate patient-reporting view summarizes, per patient, the total number of appointments, prescriptions, and medical records on file, giving staff a quick at-a-glance sense of a patient's engagement with the facility. Finally, a doctor calendar view consolidates the weekly availability and bookings of all doctors into a single view, which is useful for administrative planning and for identifying scheduling gaps or bottlenecks across departments.

6. Data Model

The relational schema underlying HMS is composed of seven primary entities, modeled through Django's object-relational mapper. Rather than accepting the framework's default behavior uniformly across every relationship, each foreign key in the schema was individually reviewed and assigned a deletion policy — cascade or set-null — based on a deliberate judgment about whether the child record's existence is entirely dependent on its parent, or whether it represents a historical fact that should persist even if the parent record is later removed.

6.1 Entity-Relationship Overview



6.2 Entity Reference

The **User** entity is the base identity record for every account in the system, whether Administrator, Doctor, or Receptionist, and carries a role attribute alongside the standard username, hashed password, and e-mail fields. The **Department** entity represents an organizational unit of the hospital and optionally designates one of its associated doctors as head of department. The **Doctor** entity extends a **User** account with clinical-specific attributes — department assignment, specialization, availability, and consultation fee — and is the anchor point for every appointment that doctor is booked into. The **Patient** entity holds demographic and contact information along with a registration date, and is the anchor point for that patient's entire appointment history. The **Appointment** entity ties a **Patient** and a **Doctor** together at a specific date and time slot, carries a status field governing its lifecycle, and is the parent record for any **Prescription** or **MedicalRecord** that results from the visit. The **Prescription** entity records the fact that a doctor issued a set of medicines for a completed appointment, and delegates the actual medicine detail to one or more **PrescriptionItem** child records, each specifying a medicine name, dosage, frequency, duration, and instructions. The **MedicalRecord** entity captures the clinical outcome of a visit — diagnosis, treatment, and test results — as a structured record tied to its originating appointment.

Entity	Key Attributes	Relationships
User	username, password (hashed), role, email, is_active	Base identity for Admin, Doctor, and Receptionist accounts
Department	name, description, head_of_department	One-to-many with Doctor; self-referential link to a Doctor as head
Doctor	user (1:1), department (FK), specialization, availability, consultation_fee	Many-to-one with Department; one-to-many with Appointment
Patient	full_name, date_of_birth, gender, contact_number, address, registration_date	One-to-many with Appointment
Appointment	patient (FK), doctor (FK), date, time_slot, status, notes	Many-to-one with Patient and Doctor; one-to-one/many with Prescription and MedicalRecord
Prescription	appointment (FK), doctor (FK), date_issued, notes	One-to-many with PrescriptionItem
PrescriptionItem	prescription (FK), medicine_name, dosage, frequency, duration, instructions	Many-to-one with Prescription
MedicalRecord	appointment (FK), diagnosis, treatment, test_results, recorded_on	Many-to-one with Appointment

6.3 Referential Integrity Rules

The deletion policy assigned to each foreign key was chosen to reflect the real-world dependency between the two entities it connects, rather than left at Django's uniform default. A PrescriptionItem is deleted in cascade when its parent Prescription is deleted, because an individual medicine line item has no independent meaning outside the prescription it belongs to. An Appointment's link to a Doctor, by contrast, is set to null rather than cascading, because the historical fact that an appointment took place must survive even if the doctor's account is later removed from the system — deleting the appointment along with the doctor would silently erase clinical history. An Appointment's link to a Patient does cascade, on the reasoning that an appointment record has no standalone value once its patient has been removed from the system entirely. Finally, the head_of_department link on Department is set to null rather than cascading when the relevant doctor is removed, because losing a department's designated head should never result in the department itself being deleted.

Relationship	On-Delete Rule	Rationale
PrescriptionItem → Prescription	CASCADE	A medicine line item has no meaning without its parent prescription
Appointment → Doctor	SET_NULL	Historical appointment records must survive removal of a doctor account
Appointment → Patient	CASCADE	Appointments are meaningless without an associated patient record
Department → Doctor (head_of_department)	SET_NULL	Removing the designated head should not delete the department itself

6.4 Notable Design Decisions

Two design decisions in particular are worth calling out because they reflect problems that are easy to get wrong in a Django project and expensive to fix after the fact. The first is the introduction of a custom User model before the very first migration was applied. Django strongly recommends this approach because swapping the User model after a project already has migration history is a substantially more disruptive operation, potentially requiring a full database reset; by introducing the role-carrying custom User model from the outset, HMS avoided that entire class of migration pain. The second is the resolution of the circular reference between Department and Doctor: a department has many doctors, but a department may also designate one particular doctor as its head. This circularity was resolved by adding the `head_of_department` field to Department in a migration issued only after the Doctor model already existed, which sidesteps the chicken-and-egg problem of two models each needing to reference the other at creation time. Finally, appointment status was deliberately modeled as a constrained choice field — Pending, Confirmed, Completed, or Cancelled — rather than a free-text field, which guarantees that every part of the system that branches on appointment status (such as gating prescription creation on Completed status) can rely on a closed, predictable set of values.

7. Roles and Access Control

Access control in HMS is role-based and enforced server-side, independent of whatever navigation links happen to be visible in a given user's interface. Three roles are defined, and each carries a distinct scope of responsibility and a distinct scope of data access.

Role	Primary Responsibilities	Access Level
Administrator	Manage departments, doctors, and receptionist accounts; oversee system-wide records and reports	Full access to all modules
Doctor	View personal appointment schedule and weekly calendar; access assigned patients' history; issue prescriptions and medical records	Restricted to own appointments, patients, and records
Receptionist	Register patients; book, modify, and manage appointments; front-desk operations	Access to patient and appointment modules only

The Administrator role carries the broadest scope of authority: administrators can create and edit departments, add and manage doctor and receptionist accounts, and view records across the entire system rather than being scoped to a single doctor's patients. The Doctor role is deliberately narrower — a doctor can see their own appointment schedule and a consolidated weekly calendar, and can access the history of patients they have been or are currently treating, but cannot see or modify another doctor's schedule or patients. The Receptionist role is narrower still, scoped specifically to the patient-registration and appointment-booking workflow that constitutes front-desk operations, without access to clinical data such as prescriptions or medical records.

Enforcement of these boundaries happens at the view layer, not merely at the template layer. Every view responsible for displaying or modifying sensitive data is wrapped in a role-checking decorator that validates the authenticated user's role before any query is executed or any response rendered; a request from a user whose role does not permit the requested action is redirected rather than served a partial or error page that might leak information about the existence or shape of the underlying data. This means that hiding a navigation link for a given role is a usability convenience, not the actual security boundary — the actual boundary is enforced independently, so that a user cannot bypass access control simply by navigating directly to a URL that their dashboard does not expose.

8. Core Workflows

8.1 Appointment Booking

The appointment-booking workflow begins when a Receptionist or Administrator selects a registered patient and an available doctor for a proposed date and time slot. Before the appointment is committed to the database, the scheduling engine queries the selected doctor's existing appointments for that date and checks for any overlap with the requested time slot; if a conflict is detected, the booking is rejected and the user is prompted to select a different slot, which is the mechanism by which the system guarantees that a doctor can never be double-booked into two overlapping appointments. Once the conflict check passes, the appointment is created with an initial status of Pending, reflecting that it has been requested but not yet formally confirmed. From there, the appointment's status is progressed by staff through Confirmed and eventually Completed as the visit actually takes place, or it may instead be marked Cancelled if the patient or the hospital cancels the visit before it occurs. This status field is not merely descriptive — it is the gate that controls whether downstream actions, such as prescription creation, are permitted for that appointment.

8.2 Prescription and Medical Record Creation

Once an appointment reaches Completed status, the doctor assigned to that appointment gains the ability to create a Prescription record for the visit. A prescription is not a single free-text field but a structured record that can contain multiple itemized medicine entries, each specifying the medicine name, dosage, frequency, duration, and any administration instructions, which allows a single prescription to accurately represent a complete, multi-drug treatment plan rather than forcing the doctor to compress that information into an unstructured note. In parallel, the same doctor may record a MedicalRecord for the same appointment, capturing the diagnosis reached, the treatment administered, and any test results relevant to the visit. Because both of these record types are anchored to a specific completed appointment rather than existing independently, the system guarantees that every prescription and every medical record can always be traced back to a verifiable clinical encounter. Once a prescription has been created, it can be exported as a formatted PDF document, giving the patient a portable, professional record of their treatment that does not require access to the hospital's internal system to be useful outside the facility.

9. Installation and Deployment Guide

The installation procedure for HMS was designed to be completed in a small number of straightforward steps, reflecting the deliberate choice to avoid dependencies on external services such as a standalone database server. The following procedure describes how to configure and run the application in a local development environment.

Step 1 — Prepare the project directory

Obtain the project source files and navigate into the project's root directory in a terminal session.

Step 2 — Create and activate a virtual environment

```
python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # macOS / Linux
```

Step 3 — Install dependencies

```
pip install -r requirements.txt
```

Step 4 — Apply database migrations

```
python manage.py migrate
```

Step 5 — Seed demonstration data

```
python manage.py seed_data
```

Step 6 — Launch the development server

```
python manage.py runserver
```

Once the development server has started, the application is reachable at `http://127.0.0.1:8000/` in a local browser. No additional configuration, such as a separate database server or message queue, is required to reach a fully functional local instance of the system.

9.1 Demonstration Accounts

The seed procedure provisions one account for each of the three roles in the system, so that every functional area can be evaluated immediately after installation without requiring manual account creation.

Role	Username	Password
Administrator	admin	admin123
Doctor (1)	doctor1	doctor123
Doctor (2)	doctor2	doctor123
Doctor (3)	doctor3	doctor123
Receptionist	receptionist	reception123

These credentials are provided strictly for demonstration and evaluation purposes and must be replaced with strong, unique credentials before any production deployment; they should never be left active on a system that holds real patient data.

9.2 Production Deployment Notes

While the default configuration is intentionally optimized for ease of local setup, several changes are necessary before the system should be considered ready for production use. A Procfile is already included in the project, declaring the web process entry point expected by platform-as-a-service hosting providers, which simplifies the initial deployment step. However, SQLite, while entirely adequate for demonstration and light single-user testing, is not architected for the concurrent write load a real hospital would generate, and a production deployment should migrate the data layer to a managed relational database such as PostgreSQL. Similarly, environment-specific secrets — including the Django `SECRET_KEY`, e-mail credentials used for appointment reminders, and the list of allowed hosts — should be externalized to environment variables rather than committed to source control, both as a security best practice and to allow the same codebase to be safely promoted across development, staging, and production environments. Finally, static assets should be collected via the `collectstatic` management command and served through a dedicated static file host or content delivery network rather than through Django's development-time static file handling, which is neither designed nor intended for production traffic.

10. Project Structure

The codebase is organized so that its directory layout mirrors the functional architecture described in Section 3, making it straightforward for a new contributor to locate the code responsible for any given piece of functionality without needing to search the entire project.

```
hospital/
├── accounts/           # Custom User model, authentication, dashboards
├── departments/        # Department CRUD
├── doctors/            # Doctor CRUD, doctor-specific views
├── patients/           # Patient CRUD
├── appointments/       # Appointment scheduling
├── records/            # Prescriptions (with inline items), Medical Records
├── hospital/           # Project settings, URL configuration
├── templates/          # All HTML templates (Bootstrap 5)
│   ├── base.html       # Base template with role-aware navbar
│   ├── registration/   # Login page
│   ├── accounts/       # Role dashboards
│   ├── departments/    # Department templates
│   ├── doctors/        # Doctor templates and calendar view
│   ├── patients/       # Patient templates
│   ├── appointments/   # Appointment templates
│   └── records/        # Prescription and medical record templates
├── static/             # Static assets (CSS, JS, images)
├── manage.py
└── requirements.txt
```

11. Management Commands

HMS extends Django's `manage.py` tooling with two project-specific management commands that support operational and demonstration needs beyond the standard framework commands.

Command	Purpose
<code>python manage.py migrate</code>	Applies pending database migrations
<code>python manage.py seed_data</code>	Populates the database with demonstration data and accounts
<code>python manage.py send_reminders</code>	Dispatches e-mail reminders for appointments scheduled for the following day
<code>python manage.py runserver</code>	Starts the local development web server

The `seed_data` command is particularly valuable for evaluation and demonstration purposes, since it removes the friction of manually creating departments, doctors, patients, and accounts before the system can be meaningfully explored. The `send_reminders` command is intended to be invoked on a recurring schedule — for example, via a daily cron job or an equivalent scheduled-task mechanism on the hosting platform — so that patients automatically receive a reminder e-mail the day before their appointment without requiring staff to track and send those reminders manually.

12. Testing Strategy

The system incorporates automated end-to-end testing using Playwright, which drives a real browser against the running application to validate that critical user journeys behave correctly from the perspective of an actual user rather than only at the level of isolated unit tests. This end-to-end coverage spans authentication flows — verifying

that each of the three roles logs in successfully and is redirected to the correct dashboard — as well as the full appointment lifecycle, including booking, the double-booking conflict check, and the progression of status from Pending through Completed. It also covers the restriction of prescription and medical-record creation to doctors, and specifically confirms that this creation is only possible once the relevant appointment has reached Completed status, along with the standard create, read, update, and delete flows for departments, doctors, and patients.

While this end-to-end coverage validates that the system behaves correctly as a whole, it is recommended that test coverage be extended with Django's built-in test framework to add unit-level validation of individual model constraints, form validation rules, and permission decorators in isolation. Unit-level tests of this kind execute far faster than browser-driven end-to-end tests and make it substantially easier to pinpoint the exact cause of a regression when one is introduced, complementing rather than replacing the existing end-to-end suite.

13. Security Considerations

Several security measures are already built into the system by virtue of the frameworks and practices it relies on, while several others are recommended as necessary hardening steps before any production deployment involving real patient data.

Passwords are never stored in plain text; Django's built-in password-hashing machinery is used for every account in the system, so that even direct access to the underlying database would not expose usable credentials. Role-based decorators restrict access to sensitive views at the server layer, independent of whatever links or buttons happen to be rendered in a given user's interface, which means that access control cannot be circumvented simply by navigating directly to a restricted URL. Django's built-in Cross-Site Request Forgery (CSRF) protection is applied to every state-changing form submission in the system, mitigating a well-known class of attack in which a malicious site attempts to trick an authenticated user's browser into submitting an unintended request. Input submitted through every form in the system is validated against Django's form and model-level constraints before being persisted, which reduces the risk of malformed, inconsistent, or maliciously crafted data entering the database.

For any deployment intended to hold real patient data, several additional steps are strongly recommended: Django's DEBUG mode should be disabled, since debug mode can expose sensitive internal details in error pages; the ALLOWED_HOSTS setting should be configured explicitly rather than left permissive, to prevent host-header manipulation attacks; and all traffic should be served exclusively over HTTPS, so that patient data and authentication credentials are never transmitted in plain text over the network.

14. Limitations and Future Enhancements

14.1 Current Limitations

Three limitations of the current system are worth stating candidly. First, SQLite, while entirely sufficient for demonstration purposes and light single-user testing, is not designed for the concurrent write workload that a real, multi-user hospital deployment would generate, and would need to be replaced before the system could reliably serve a busy facility. Second, the system does not currently expose a REST or GraphQL API, which means that no mobile application or third-party system can currently integrate with it programmatically; all interaction is presently limited to the server-rendered web interface. Third, reporting is currently limited to a per-patient

summary view, and the system does not yet provide a consolidated, facility-wide analytics dashboard covering metrics such as appointment volume trends, department utilization, or revenue derived from consultation fees.

14.2 Recommended Enhancements

A number of enhancements would meaningfully extend the system's production readiness and operational value. Introducing a REST API layer, for example using Django REST Framework, would allow the system to support a companion mobile application or to integrate with third-party services such as insurance-claims processors or laboratory-information systems. Migrating the production database from SQLite to PostgreSQL would resolve the concurrency limitation described above and would also unlock more advanced querying and indexing capabilities as the dataset grows. Adding an audit trail — a persistent change log — for sensitive record modifications, particularly to medical records and prescriptions, would provide the kind of accountability and traceability that is typically expected of clinical record systems, allowing any change to be attributed to a specific user at a specific time. Implementing two-factor authentication for Administrator and Doctor accounts would substantially raise the bar against credential-based attacks on the accounts with the broadest data access. Finally, introducing a consolidated analytics dashboard covering appointment volume, department-level utilization, and consultation-fee revenue would give hospital management the kind of aggregate operational visibility that individual patient-level reporting cannot provide.

15. Conclusion

The Hospital Management System, as documented in the preceding sections, delivers a complete, role-aware Django application covering the full operational lifecycle of a hospital or clinic — from patient registration, through appointment scheduling with automated conflict prevention, to prescription issuance and structured medical record keeping. Its modular application structure keeps each functional domain cleanly separated and independently maintainable, while deliberate database design choices — including differentiated cascade-versus-nullify rules on individual foreign keys and a purpose-built custom user model introduced ahead of the first migration — keep the underlying data layer robust, internally consistent, and resistant to the kind of accidental data loss that a less carefully considered schema would be prone to.

Taken as a whole, the system represents a solid and well-reasoned technical foundation. It is not, in its current form, a finished production hospital information system — the limitations enumerated in Section 14 are real and should be treated as a concrete roadmap rather than a formality — but it demonstrates disciplined engineering practice throughout its architecture, its data model, and its access-control design, and it provides a credible, extensible base from which a production-grade deployment could be built.

16. Appendix — Glossary of Terms

Term	Definition
MVT	Model-View-Template — Django's architectural pattern separating data, logic, and presentation
ORM	Object-Relational Mapping — Django's abstraction layer for interacting with the relational database
CRUD	Create, Read, Update, Delete — the four basic operations of persistent storage

Term	Definition
RBAC	Role-Based Access Control — restricting system functionality according to a user's assigned role
CSRF	Cross-Site Request Forgery — an attack class mitigated by Django's built-in token protection
Foreign Key (FK)	A field that establishes a relationship by referencing the primary key of another table

SCREENSHOT

ADMIN-PANEL

The top screenshot displays the login interface of the Hospital Management System. It features a blue header with the system name and a central login form. The form includes fields for 'Username' (pre-filled with 'admin') and 'Password' (pre-filled with 'admin123'), both with eye icons for toggling visibility. A blue 'Sign In' button is positioned below the password field. The browser's address bar shows the URL '127.0.0.1:8000/login/'.

The bottom screenshot shows the Admin Dashboard. It has a blue header with the system name and a navigation menu. The dashboard includes four summary cards: 'Total Patients' (15), 'Total Doctors' (3), 'Today's Appointments' (3), and 'Pending Appointments' (4). Below these cards is a 'Quick Actions' section with buttons for 'Manage Departments', 'Manage Doctors', 'Manage Patients', and 'Manage Appointments'. The browser's address bar shows the URL '127.0.0.1:8000/dashboard/admin/'.

East-Pakistan to Bangladesh

Departments - HMS

Ask Gemini

127.0.0.1:8000/departments/

Arqam Hussain — H...Cisco Networking A...Portfolio Ideas | Port...LockedIn AI - Profes...Latest Remote & Of...Freelancer Profile --Cisco Networking A...All Bookmarks

Hospital Management

DashboardDepartmentsDoctorsPatientsAppointmentsPrescriptionsRecordsReportsCalendaradmin

Departments

Add Department

Name	Head of Department	Actions
General Medicine	—	EditDelete
Pediatrics	Dr. Mike Brown	EditDelete
Test Dept PW	Dr. Mike Brown	EditDelete
Test Dept Playwright	Dr. John Smith	EditDelete
adas	Dr. John Smith	EditDelete

Hospital Management System © 2026

East-Pakistan to Bangladesh

Doctors - HMS

Ask Gemini

127.0.0.1:8000/doctors/

Arqam Hussain — H...Cisco Networking A...Portfolio Ideas | Port...LockedIn AI - Profes...Latest Remote & Of...Freelancer Profile --Cisco Networking A...All Bookmarks

Hospital Management

DashboardDepartmentsDoctorsPatientsAppointmentsPrescriptionsRecordsReportsCalendaradmin

Doctors

Add Doctor

Search...

All Departments

All Specializations

Filter

Name	Department	Specialization	Fee	Status	Actions
John Smith	—	Interventional Cardiology	\$150.00	Active	ViewEditDelete
Sarah Johnson	—	Neurology	\$200.00	Active	ViewEditDelete
Mike Brown	Pediatrics	Pediatric Medicine	\$100.00	Active	ViewEditDelete

Hospital Management System © 2026

East-Pakistan to Bangladesh | Patients - HMS

127.0.0.1:8000/patients/

Ask Gemini

Arqam Hussain — H...Cisco Networking A...Portfolio Ideas | Port...LockedIn AI - Profes...Latest Remote & Of...Freelancer Profile --Cisco Networking A...

+

Hospital Management

Dashboard

Departments

Doctors

Patients

Appointments

Prescriptions

Records

Reports

Calendar

admin

Patients

Register Patient

Search by name, phone, o

Search

ID	Name	Phone	Gender	Registered	Actions
1	Alice Williams	5551111111	Female	Jun 30, 2026	View Edit Delete
9	Ava Martin	5559999999	Female	Jun 30, 2026	View Edit Delete
10	Daniel Garcia	5550000000	Male	Jun 30, 2026	View Edit Delete
2	David Miller	5552222222	Male	Jun 30, 2026	View Edit Delete
7	Emma White	5557777777	Female	Jun 30, 2026	View Edit Delete
4	James Anderson	5554444444	Male	Jun 30, 2026	View Edit Delete
8	Lucas Harris	5558888888	Male	Jun 30, 2026	View Edit Delete

Type here to search

29°C Mostly cloudy

12:09 AM 7/2/2026

East-Pakistan to Bangladesh | Appointments - HMS

127.0.0.1:8000/appointments/

Ask Gemini

Arqam Hussain — H...Cisco Networking A...Portfolio Ideas | Port...LockedIn AI - Profes...Latest Remote & Of...Freelancer Profile --Cisco Networking A...

+

Hospital Management

Dashboard

Departments

Doctors

Patients

Appointments

Prescriptions

Records

Reports

Calendar

admin

Appointments

Schedule Appointment

Search patient...

mm/dd/yyyy

All Doctors

All Departments

All Status

Filter

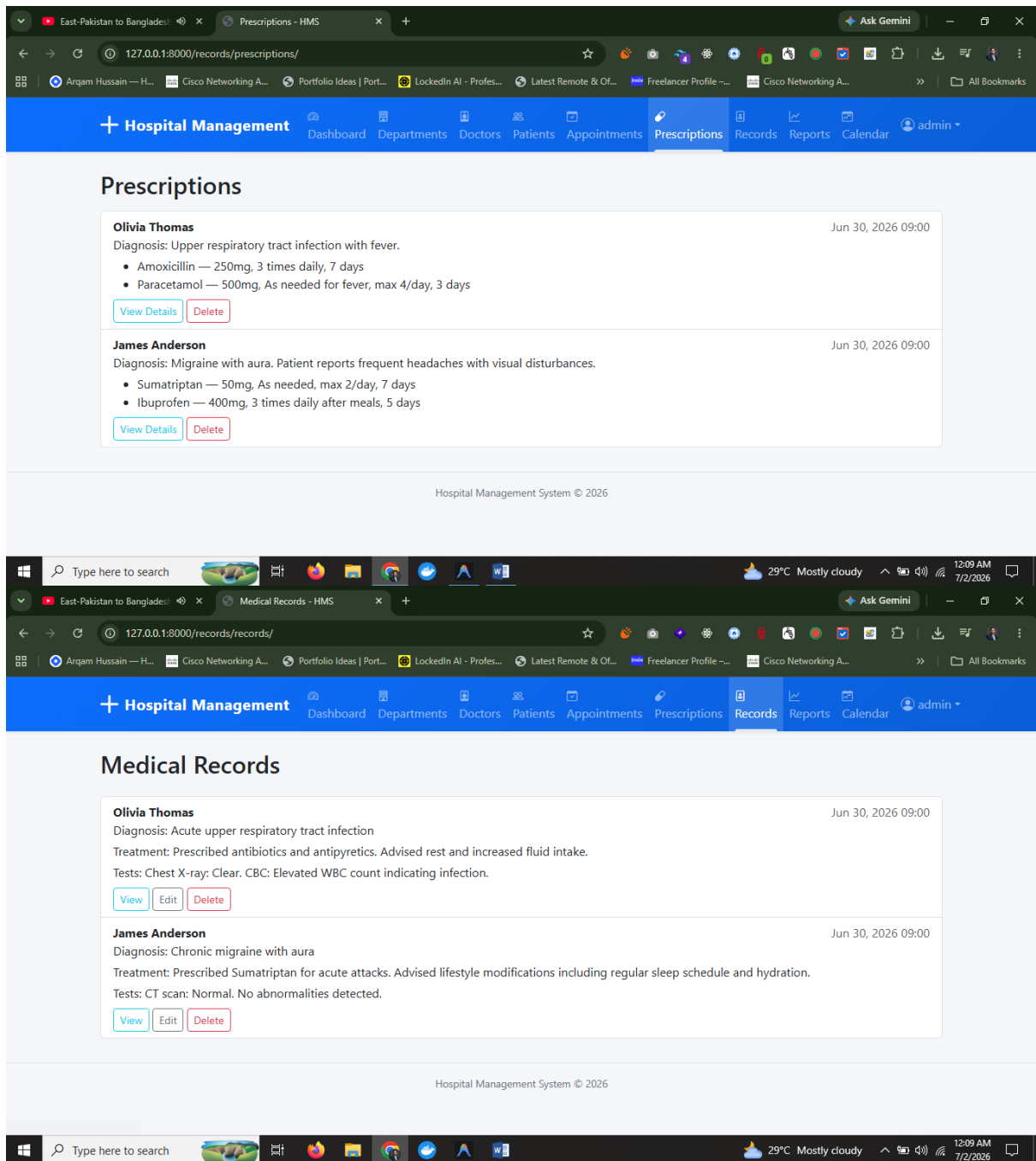
Patient	Doctor	Date	Time	Status	Actions
Daniel Garcia	Dr. John Smith	July 2, 2026	10:00	Pending	View Edit
Ava Martin	Dr. Mike Brown	July 1, 2026	14:00	Pending	View Edit
Lucas Harris	Dr. Sarah Johnson	July 1, 2026	11:00	Pending	View Edit
Emma White	Dr. John Smith	July 1, 2026	09:00	Pending	View Edit
Sophia Taylor	Dr. Sarah Johnson	June 30, 2026	23:00	Confirmed	View Edit
James Anderson	Dr. Sarah Johnson	June 30, 2026	14:00	Completed	View Edit
William Jackson	Dr. Mike Brown	June 30, 2026	10:30	Cancelled	View Edit

Type here to search

29°C Mostly cloudy

12:09 AM 7/2/2026

Prepared by Arqam Hussain | Page 18 of 22



East-Pakistan to Bangladesh | Patient Report - HMS

127.0.0.1:8000/records/patient-report/

Ask Gemini

Arqam Hussain — H...Cisco Networking A...Portfolio Ideas | Port...LockedIn AI - Profes...Latest Remote & Of...Freelancer Profile --Cisco Networking A...

Hospital Management

DashboardDepartmentsDoctorsPatientsAppointmentsPrescriptionsRecordsReportsCalendaradmin

Patient Summary Report

15
Total Patients

10
Total Appointments

2
Prescriptions

2
Medical Records

Patient List

ID	Name	Phone	Gender	Appts	Scripts	Records
1	Alice Williams	5551111111	Female	1	0	0
9	Ava Martin	5559999999	Female	1	0	0
10	Daniel Garcia	5550000000	Male	1	0	0
2	David Miller	5552222222	Male	1	0	0
7	Emma White	5557777777	Female	1	0	0
4	James Anderson	5554444444	Male	1	1	1

East-Pakistan to Bangladesh | Doctor Availability Calendar - HMS

127.0.0.1:8000/doctors/calendar/

Ask Gemini

Arqam Hussain — H...Cisco Networking A...Portfolio Ideas | Port...LockedIn AI - Profes...Latest Remote & Of...Freelancer Profile --Cisco Networking A...

Hospital Management

DashboardDepartmentsDoctorsPatientsAppointmentsPrescriptionsRecordsReportsCalendaradmin

admin

Admin

Logout

Doctor Availability Calendar

Dr. John Smith

Specialization: Interventional Cardiology

Time: 09:00 — 17:00

Fee: \$150.00

MoTuWeThFrSaSu

Book

Dr. Sarah Johnson

Specialization: Neurology

Time: 10:00 — 18:00

Fee: \$200.00

MoTuWeThFrSaSu

Book

Dr. Mike Brown

Specialization: Pediatric Medicine

Time: 09:00 — 15:00

Fee: \$100.00

MoTuWeThFrSaSu

Book

Hospital Management System © 2026

East-Pakistan to Bangladesh | Doctor Availability Calendar - HMS

127.0.0.1:8000/doctors/calendar/

Ask Gemini

Arqam Hussain — H...Cisco Networking A...Portfolio Ideas | Port...LockedIn AI - Profes...Latest Remote & Of...Freelancer Profile --Cisco Networking A...

Hospital Management

DashboardDepartmentsDoctorsPatientsAppointmentsPrescriptionsRecordsReportsCalendaradmin

admin

Admin

Logout

Doctor Availability Calendar

Dr. John Smith

Specialization: Interventional Cardiology

Time: 09:00 — 17:00

Fee: \$150.00

MoTuWeThFrSaSu

Book

Dr. Sarah Johnson

Specialization: Neurology

Time: 10:00 — 18:00

Fee: \$200.00

MoTuWeThFrSaSu

Book

Dr. Mike Brown

Specialization: Pediatric Medicine

Time: 09:00 — 15:00

Fee: \$100.00

MoTuWeThFrSaSu

Book

Hospital Management System © 2026

Prepared by Arqam Hussain | Page 20 of 22

DOCTOR 1-PANEL

East-Pakistan to Bangladesh | Doctor Dashboard - HMS | Ask Gemini

127.0.0.1:8000/dashboard/doctor/

Arqam Hussain — H... Cisco Networking A... Portfolio Ideas | Port... LockedIn AI - Profes... Latest Remote & Of... Freelancer Profile -... Cisco Networking A... All Bookmarks

Hospital Management | Dashboard | My Appointments | Patients | Scripts | Records | Calendar | John

Doctor Dashboard

Today's Appointments
1

Total Appointments
4

Unique Patients
4

Today's Appointments

Time	Patient	Status	Action
09:00	Emma White	Pending	View

Hospital Management System © 2026

Type here to search | 29°C Mostly cloudy | 12:14 AM 7/2/2026

DOCTOR2-PANEL

East-Pakistan to Bangladesh | Doctor Dashboard - HMS | Ask Gemini

127.0.0.1:8000/dashboard/doctor/

Arqam Hussain — H... Cisco Networking A... Portfolio Ideas | Port... LockedIn AI - Profes... Latest Remote & Of... Freelancer Profile -... Cisco Networking A... All Bookmarks

Hospital Management | Dashboard | My Appointments | Patients | Scripts | Records | Calendar | Sarah

Doctor Dashboard

Today's Appointments
1

Total Appointments
4

Unique Patients
4

Today's Appointments

Time	Patient	Status	Action
11:00	Lucas Harris	Pending	View

Hospital Management System © 2026

Type here to search | 29°C Mostly cloudy | 12:15 AM 7/2/2026

DOCTOR3-PANEL

East-Pakistan to Bangladesh...Doctor Dashboard - HMSarqam66/digitech_final_testAsk Gemini

127.0.0.1:8000/dashboard/doctor/

Arqam Hussain — H...Cisco Networking A...Portfolio Ideas | Port...LockedIn AI - Profes...Latest Remote & Of...Freelancer Profile --Cisco Networking A...All Bookmarks

Hospital Management

DashboardMy AppointmentsPatientsScriptsRecordsCalendar

Mike

Doctor Dashboard

Today's Appointments1

Total Appointments3

Unique Patients3

Today's Appointments

Time	Patient	Status	Action
14:00	Ava Martin	Pending	View

Hospital Management System © 2026

Type here to search

29°C Mostly cloudy12:18 AM7/2/2026

RECEPTIONIST-PANEL

East-Pakistan to Bangladesh...Receptionist Dashboard - HMSarqam66/digitech_final_testAsk Gemini

127.0.0.1:8000/dashboard/receptionist/

Arqam Hussain — H...Cisco Networking A...Portfolio Ideas | Port...LockedIn AI - Profes...Latest Remote & Of...Freelancer Profile --Cisco Networking A...All Bookmarks

Hospital Management

DashboardPatientsAppointmentsDoctorsCalendar

Receptionist Dashboard

Today's Appointments3

Total Patients15

Pending Appointments4

Today's Schedule

Time	Patient	Doctor	Status
09:00	Emma White	Dr. John Smith	Pending
11:00	Lucas Harris	Dr. Sarah Johnson	Pending
14:00	Ava Martin	Dr. Mike Brown	Pending

Register New Patient

Schedule Appointment

Add new password?
1 127.0.0.1 receptionist
Never Not now Add

Type here to search

29°C Mostly cloudy12:17 AM7/2/2026